Ders 1 Genişletilmiş Notu: Nesne Yönelimli Yaklaşım ve Java'ya Giriş

Ders 1 Genişletilmiş Notu: Nesne Yönelimli Yaklaşım ve Java’ya Giriş

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Yeri: Öncesi, Sonrası ve İki Tarafı
- 2. Nesne Yönelimli Yaklaşım Neden Gerekli?
- 3. UML ve Kaynak Kitaplar
- 4. Dersin Java Ağırlığı ve Java’da Olmayan Kavramlar
- 5. Asıl Zor Olan Ne? Analiz ve Tasarım
- 6. Dönem Boyunca Görülecek Konular
- 7. Neden Java?
- 8. Java Sürümleri: SE, ME ve EE
- 9. Tek Sözdizimi, Güçlü Dokümantasyon
- 10. Java Programı Nasıl Derlenir ve Çalışır?
- 11. Java’nın Gelişimi
- 12. Geliştirme Ortamları, UML Araçları ve Plugin Kavramı
- 13. IntelliJ IDEA Üzerinde Kısa Bir İnceleme

1. Dersin Yeri: Öncesi, Sonrası ve İki Tarafı

Hoca, nesne yönelimli programlamanın (object-oriented programming) tek başına duran bir konu olmadığını söyleyerek başladı. Bu dersin bir öncesi ve bir sonrası vardır.

Dersin öncesi yapısal programlama ve algoritma bilgisidir. Bu bilgi C ya da Pascal ile kazanılmış olabilir. Önemli olan, verilen bir problemi, örneğin bir dizinin sıralanmasını, algoritmik olarak çözebilmektir. Bu ders o bilgileri kapsar, yani onların üzerine kurulur.

Dersin sonrası ise veri tabanı, sistem analizi, web programlama ve mobil programlama gibi konulardır. Güncel hayattaki birçok uygulamanın temeli bu derste atılır. Hoca bunun gerekçesini şöyle açıkladı: Yapısal programlamayla çözmesi çok güç olan problemlerin nesne tabanlı yaklaşımla daha rahat çözüleceği iddia edilir. Bu yaklaşım bugün çok yaygın kullanıldığı için hocaya göre iddia doğrudur. Yeni bir programlama metodolojisi geliştirilene kadar da kullanılmaya devam edecektir.

Dersin iki tarafı vardır. Teorik taraf **nesneye dayalı kavramlar**dır. Uygulama tarafı ise **nesneye dayalı programlama**dır. Eski eğitim planında bunlar iki ayrı dersti. Yeni planda ikisi tek derste, daha sıkıştırılmış olarak verilir.

1.1 Referans Bakış Açısı

Bu derste önceki derslerden farklı olarak olaya işaretçi (pointer) ya da referans (reference) bakış açısıyla yaklaşmak gerekir. Java’da işaretçiler doğrudan yoktur. Ama bellekte gösterilen bir noktanın, yani bir referans noktasının bulunduğunu sürekli aklımızda tutmamız gerekir.

İlkel veri tipleri (primitive) üzerinden çok fazla ilerlemeyeceğiz, çünkü onlar yapısal programlamanın konusudur.

1.2 Diğer Derslerle Bağlantı

Hoca, dönemdeki derslerin birbiriyle ilintili olduğunu vurguladı. Hesaplama kuramında öğrenilen bir şeyin kodu C'de ya da Java'da yazılabilir. Veri yapılarında öğrenilen konular da böyledir. Java'da veri yapılarının hazır kodlarını göreceğiz ve hazır framework üzerinden çalışacağız. Yani bir problemi hem veri yapılarını kullanarak hem de nesne tabanlı yaklaşımla çözmeyi öğreneceğiz.

> **Hocanın vurgusu:** Veri yapıları dersinde C ile yazdığınız bir şeyi burada Java'da da yazarsanız hem Java bilginiz hem veri yapıları bilginiz gelişir.

Hoca ayrıca anlaşılmayan bir konunun kayıttan ya da başka bir hocanın kaydından tekrar izlenebileceğini söyledi. Yüz yüze derste kaçırılan bir konuyu telafi etmek zordur. Uzaktan eğitimin avantajı, aynı konuyu tekrar açıp izleyebilmektir.

2. Nesne Yönelimli Yaklaşım Neden Gerekli?

2.1 Fonksiyonlar ve Dosyalar Yetmiyor

Şimdiye kadar bir program yazarken yaptığımız en ileri şey, programı fonksiyonlara bölmekti. Ya da programı farklı C dosyalarına parçalayıp `include` ve `.h` dosyalarıyla birleştiriyorduk. Fakat bu, projeyi yeterince modüler hale getirmeye ve yönetmeye yetmez. Bunun için zamanında farklı bir şey düşünülmüştür: nesne yönelimli programlama.

Bu yaklaşımda önce nesne (object) ve sınıf (class) gibi temel kavramlar oluşturulmuştur. Ardından sınıflar arasındaki ilişkiler tanımlanmıştır. Bu ilişkilerden ve sınıflardan yola çıkılarak da **nesneye dayalı analiz ve tasarım** kavramı ortaya atılmıştır.

Temel soru şudur: Bir problemi kodlamadan önce, bu problemde hangi nesneler olabilir ve bu nesneler arasındaki ilişkiler nasıl olabilir? Bir algoritmayı kodlamadan önce akış şemasını (flowchart) çizer ya da kafanızda hangi değişkenlerin hangi türde olacağına dair bir taslak oluşturursunuz. Burada da aynısı yapılır, ama taslak nesneler üzerinden kurulur. Olabildiğince ilkel veri tiplerinden, yani tamsayılardan, karakterlerden ve ondalıklı sayılardan kaçınacağız. Nesnesel düşüneceğiz ve nesneler arasındaki ilişkileri doğru tasarlamaya çalışacağız.

2.2 Kazançlar ve Web Sitesi Örneği

Bu yaklaşım büyük bir projede dört açıdan kazanç sağlar:

- Zaman
- Yönetim
- Bakım
- Anlaşılabilirlik

Hoca bunu bir web sitesiyle örnekledi. Sitenin altında yüzlerce sayfa olduğunu düşünelim. Bu sayfaların her biri aslında bir nesneyle ilgili bir işlem yapar. Biri kitap ekler, biri kitap siler, biri kitap oluşturur, bir diğeri yazarla ilgili işlem yapar. Kitap ile yazar arasındaki ilişki ise ayrıca tasarlanır. Bunların hepsinin temeli nesne kavramı ve sınıflar arası ilişkilerdir.

Bunu yapısal programlamayla da yapabilir miydik? Elbette. Orada da dosya sistemleri ve veri tabanı bağlantıları vardır. Ama yönetmesi, bakımı ve anlaşılması çok daha zordur.

2.3 Yaşayabilir Yazılım ve Robot Benzetmesi

Yazılım projelerinin en önemli özelliklerinden biri yazılımın **yaşayabilir** olmasıdır. Ortaya koyduğunuz yazılım ne kadar çok kullanılıyorsa o kadar iyidir. Ama gereksinimler sürekli arttığı için yazılımı sürekli güncellemeniz gerekir. Bu sırada herkes programı kullanmaya devam edebilmelidir. "Sistemi kapattık, yarın açacağız" gibi bir durum olmamalıdır.

Tasarım öyle olmalıdır ki hem kimseyi rahatsız etmeyelim hem de programda çok ciddi değişiklikler yapmayalım. Hoca bunu bir robota parça takmaya benzetti: Yeni bir modülü getirip

sisteme taktığımızda sistem kendini ona uyarlar ve yeni işlevler ortaya çıkar. Belki küçük bir kısmı devre dışı bırakır ya da bakıma alırız, ama genelde ciddi bir değişiklik yapmayız.

2.4 Donanıma Yakınlık ve Soyutlama

Yapısal programlamada bu pek mümkün değildir. C ya da Pascal ile yazılmış bir sistemde bir değişkenin türünü değiştirdiğinizde neredeyse her yerde bir şeyler değiştirmeniz gerekir. Hocaya göre bunun sebebi, C ve Pascal gibi dillerin donanıma daha yakın, soyutlamaya daha uzak olmasıdır. Donanımla birebir iç içe değillerdir, ama donanıma bağımlı gibi düşünülürler. Nesne yönelimli tasarımda ise kendimizi donanımdan ayırıp olabildiğince soyut düşünmemiz gerekir.

3. UML ve Kaynak Kitaplar

3.1 Unified Modeling Language

Nesne nedir, sınıf nedir, aralarındaki ilişkiler nedir sorularının ortak bir lisanı vardır: **Unified Modeling Language (UML)**, yani birleşik modelleme dili. Tasarım aşamasındaki modelleme bu dille yapılır, kod daha sonra yazılır.

Hatta öyle araçlar vardır ki modeli UML ile oluşturup "code generate" dersiniz. Araç sizin adınıza boş bir Java kodu iskeleti üretir: Sınıfı yazar, altında metotları oluşturur. Yazılım mühendisliği süreçleri de zaten bu şekilde ilerler.

3.2 Nesne Tabanlı Tasarım İçin Kitaplar

Hoca, derste anlatılanların detaylarını okumanın çok şey katacağını söyledi. Yazılımla ilgilenecek kişilerin terminolojik konuşması, "sokak ağzıyla" konuşmaması gerekir. Bu da yalnız derste verilenlerle olmaz. Derste eksik kalmış bir kelimenin ya da iki ders arasındaki kopukluğun giderilmesi için bir kitap takip etmek iyi olur.

Bir nokta özellikle vurgulandı: Nesne tabanlı programlamanın var olması tek başına problemlerimizi çözmez. Algoritmalarda analiz ve verimlilik nasıl önemliyse burada da doğru analiz ve tasarım yapmak gerekir. Önerilen kitaplar:

- **Martin Fowler, *UML Distilled*:** Özet bir UML kitabı. Hoca, "distilled" sözcüğünü "rafine edilmiş" diye açıkladı. Fowler'ı yazılım mühendisliğinde üstat sayılan kişilerden biri olarak tanıttı.
- **Martin Fowler, *Analysis Patterns* ve *Refactoring*:** Aynı yazarın nesne tabanlı analiz ve tasarıma yönelik diğer kitapları.
- **Özcan Acar, *Java Tasarım Şablonları ve Yazılım Mimarileri*:** Pusula Yayınları'ndan çıkan, turuncu kapaklı Türkçe bir kitap. Hoca bunun konunun biraz daha ileri versiyonu olduğunu söyledi.
- ***Tasarım Desenleri ve Mimarileri (C#, Java, C++ Örnekleri)*:** Aynı yayınevinden Türkçe bir kitap.
- **Aykut Taşdelen, *C++, Java ve C#'ta UML ve Design Patterns*:** Hoca yazarı Türkiye'de bu konunun iyilerinden biri olarak tanıttı ve adının da araştırılmasını önerdi.

Hoca bu kitapların bu dersten geçmek için değil, meslek için önerildiğini açıkça belirtti. Bunlardan soru sormayacağını da söyledi. Mevcut Java bilgisinin üzerine bir şeyler katarlar ve nesne yönelimli analiz ve tasarım için gereken bilgileri içerirler. İnternette de "object-oriented concepts", "object-oriented paradigms", "object-oriented methods" gibi aramalarla üniversitelerin ders kaynaklarına ve eğitim sitelerine ulaşılabilir.

> **Hocanın vurgusu:** Bu iş okuryazarlık gerektirir. Bu ders ve sonraki dersler size meslek hayatınızda iş yapmanızı sağlayacak derslerdir. Nesne yönelimli mantığı bilmeyen birinin, çok donanım tarafında ya da başka bir alanda çalışmıyorsa, piyasada zorlandığını görüyorum.

3.3 Java İçin Kitaplar

Dersin Java tarafı için şu kitaplar anıldı:

- ***Core Java***
- **Deitel, *Java How to Program***
- **Daniel Liang'ın Java kitabı (*Introduction to Java Programming*):** Pearson'dan çıkar. Hocaya göre en çok satan ve en çok kullanılan Java kitaplarından biridir ve detaylıdır. Java öğrenmek isteyen çoğu kişinin başvurduğu bir kaynaktır.
- ***Absolute Java***

Hoca bu kitaplardan bazılarının "early" ve "late" diye iki farklı versiyonu olduğunu da söyledi. Bir de Türkçe bir Java kitabından bahsetti, ama kitap o an yanında değildi.

4. Dersin Java Ağırlığı ve Java'da Olmayan Kavramlar

Dersin adı nesne yönelimli programlamadır, ama uygulama olarak Java'ya ağırlık verilir. Teorik kısımda gördüğümüz nesne yönelimli kavramları hem uygulama saatlerinde hem derste Java üzerinde uygulayacağız.

Nesne yönelimli programlama kavramları içinde olup Java'da doğrudan olmayan bazı şeyler vardır. Hoca iki örnek verdi: **operatör aşırı yükleme (operator overloading)** ve **çoklu kalıtım (multiple inheritance)**. Bunlar slaytlarda yer almaz. Ama bu kavramların bilinmesi gerekir ve başka dillerde uygulamaları vardır. Bu yüzden hoca yeri geldikçe C#, C++ ya da Python'dan örnekler vereceğini söyledi.

> **Ders sonrası not:** Hoca çoklu kalıtımın "belki Java'nın son versiyonunda olabileceğini, bakmadığını" söyledi. javac 25.0.4.1 ile denendi: Sınıflar arası çoklu kalıtım yine yoktur. `class C extends A, B { }` satırı `error: '{' expected` hatası verir.

Grafik arayüz (GUI) konusu ders notlarında çok detaylı değildir. Hoca bu konuyu ek kaynaklar ve uygulamalarla desteklemeyi planladığını söyledi.

5. Asıl Zor Olan Ne? Analiz ve Tasarım

Hoca şu soruyu sordu: Bu derste zor olan nedir? Java'yı öğrenmek mi, Java'yı kodlamak mı? Cevabı kademeli olarak verdi.

5.1 Problemi Düşünmek

Hocaya göre en zoru problemi düşünmektir, yani bir problem ya da ürün üretmektir. Herkes tarafından benimsenecek, tutacak ve yaşayan bir yazılım olacak bir ürün düşünmek gerçekten zordur ve yaratıcı düşünmeye saygı duymak gerekir. Bir sürü proje ortaya atılır ve destek alır, ama çok azı yaşar.

Hoca E-Devlet örneğini verdi. Bir yazılım mühendisliği dersinde birkaç yıl önce sınıfa "kaç kişi E-Devlet kullanıyor?" diye sorulmuş, cevap belki %30 olmuştu. Hoca bugün aynı anketin belki %60-80 çıkacağını tahmin etti. Bir yazılımın yaşaması o ilk düşünce aşamasında başlar.

5.2 Geliştirme Adımları

Hoca genel uygulama geliştirme mimarisini şu adımlarla özetledi:

1. **Problem:** Çözülecek problem belirlenir.
2. **Analiz:** Probleme dair analiz yapılır. Bu, yapısal programlamada değişkenleri tespit etmeye, yani "dizi mi, matris mi kullanacağız?" sorusuna benzer. Burada olaya nesnesel bakılır.
3. **Tasarım:** Tespit edilen nesneler ve aralarındaki ilişkiler belli bir şablona oturtulur.
4. **Gerçekleme (implementation):** Tasarım herhangi bir nesne tabanlı dille kodlanır.
5. **Test ve bakım:** İstenen çıktıların ne kadar doğru olduğuna bakılır. Sonra uygulamaya geçilir.
6. **Güncelleme:** Yazılım yayına girdikten sonra güncellemeler gelir.

5.3 Kitap Örneği: Yapısal Düşünmenin Maliyeti

Hoca analiz aşamasındaki bir hatanın sonucunu kitap örneğiyle gösterdi.

Bir kitap nesnesi olacak, ama biz bunu nesne olarak tasarlamak yerine doğrudan dört değişken koyduk:

- Kitabın kimliği (ID)
- Kitabın adı
- Sayfa sayısı
- Yazar adı

Bu yapısal düşünmedir. Bu dört değişkenle yola çıktık, tasarımı yaptık, programı kodladık. İleride "kitaba bir de tür alanı eklemen lazım" dediler. Yapmanız gereken, bütün kodları açıp tek tek değiştirmektir.

Oysa ilk analiz aşamasında bunu nesnesel olarak yapsaydık tek bir yer değişecekti:

```
Kitap
  id          : tamsayı
  ad          : metin
  sayfaSayisi : tamsayı
  yazarAdi    : metin
  tur         : metin      <- sonradan eklenen tek alan
```

Bu durumda maliyet, nesnenin içine bir alan eklemek ve küçük parçaları değiştirmek kadar olur. Yapısal düşünürsek ise yazılımı yeniden yazmak neredeyse daha az maliyetli hale gelir.

5.4 Sınıflar Arası İlişkiler ve Tasarım Şablonları

Tasarımda nesneler ve sınıflar arasındaki ilişkiye çok dikkat etmek gerekir. İlerleyen haftalarda kalıtım (inheritance), çok biçimlilik (polymorphism), bileşim (composition) ve toplama (aggregation) görülecektir. Bunlar arasında farklar vardır. Toplama yerine bileşim yapınca ne olduğu, ne olmadığı önemlidir.

Bazı problemlerde bu kavramlar yetmez. O zaman kavramları bir üst seviyeye taşıyan **tasarım şablonları (design patterns)** konusuna girilir. Bu konu henüz bölümde ayrı bir ders olarak yoktur. Ama piyasanın, yani yazılım dünyasının beklediği bir şeydir. Hoca bir kısmının yazılım mühendisliği dersinde anlatıldığını zannettiğini söyledi.

6. Dönem Boyunca Görülecek Konular

Ders iki koldan yürür: Bir taraftan Java, bir taraftan nesne yönelimli kavramlar. Anlatılan bir kavram uygulamaya dökülür. Hoca, konuları sırayla geçerken birçoğu için bir ön örnek verdi.

6.1 Java'ya Giriş, Nesne ve Sınıf

Önce programlamaya yeniden giriş gibi Java'nın genel özeti görülecek. Sonra nesne tabanlı programlamanın en önemli iki kavramına, nesne ve sınıfa geçilecek. Her nesnenin belli **özellikleri** (durumu) ve bu özellikleri kullandığı **davranışları** (metotları) vardır. Hoca insan ve hayvan nesnelerini örnek gösterdi: Hepimizin özellikleri ve davranışları vardır. Aynı düşünce programlamada ve nesne yönelimli tasarımda kullanılır.

6.2 UML Sınıf Şemaları ve Diğer UML Diyagramları

Sınıflar arasındaki ilişkiler UML sınıf şemalarıyla gösterilir. UML çok geniş bir konudur. Hoca 12-13 farklı diyagram türü olduğunu hatırladığını söyledi. Etkileşim (interaction) diyagramı gibi türlerin bir kısmı sistem analizi, bir kısmı yazılım mühendisliği dersinde görülür. Veri tabanı diyagramlarıyla ilgili kısımlar da buna dahildir.

Kullanım durumu (use case) diyagramı için hoca hastane yazılımı örneğini verdi. Hastanede doktorun, hastanın ve yönetimin rolleri farklıdır. Programı kimlerin hangi rollerle nasıl kullanacağı bu diyagramla gösterilir.

6.3 İlkel Tipler, Sarmalayıcı Sınıflar ve Parametre Geçişi

Bütün nesneler sonuçta temel veri tiplerinden oluşur. Kitap nesnesinin kimliği tamsayı, adı metin (string), sayfa sayısı tamsayı ya da `byte` olabilir. Bunlara ilkel tipler (primitive) denir. İlkel tiplerin bir de sarmalayıcı sınıfları (wrapper) vardır.

Burada parametre geçişi de görülecek: değerle çağırma (call by value) ve referansla çağırma (call by reference). Hoca C'de gerçek anlamda referansla çağırma olmadığını, görüntünün onu andırdığını ama gerçekte olanın değerle çağırma olduğunu hatırlattı. Java'da ise "gerçek bir call by reference mekanizması" olduğunu, takas (swap) işleminin de wrapper sınıfları gerektirdiği için C'den daha karışık olduğunu söyledi.

> **Ders sonrası not:** Hoca Java'da gerçek bir call by reference olduğunu söyledi; doğrusu Java'da parametreler her zaman değerle geçer. Nesne parametresinde kopyalanan değer referanstır. Bu yüzden metot içinden nesnenin alanı değişir, ama parametreye yeni nesne atamak çağıranın değişkenini değiştirmez. Takasta da `Integer` kullanmak yetmez, kendi yazdığımız değiştirilebilir bir sınıf gerekir. Aşağıdaki program javac ile denendi.

```
public class ParametreGecisi {
    static class Kitap {
        String ad;
        Kitap(String ad) { this.ad = ad; }
    }

    static class Kutu {
        int deger;
        Kutu(int deger) { this.deger = deger; }
    }

    static void degistir(Kitap k) {
        k.ad = "Degisti";       // cagiranin gosterdigi nesne degisir
        k = new Kitap("Yeni");  // yalniz yerel kopya degisir
    }

    static void swapIlkel(int a, int b) {
        int t = a; a = b; b = t;
    }

    static void swapInteger(Integer a, Integer b) {
        Integer t = a; a = b; b = t;
    }

    static void swapKutu(Kutu a, Kutu b) {
        int t = a.deger; a.deger = b.deger; b.deger = t;
    }

    public static void main(String[] args) {
        Kitap kitap = new Kitap("Eski");
        degistir(kitap);
        System.out.println("kitap.ad = " + kitap.ad);

        int x = 1, y = 2;
        swapIlkel(x, y);
        System.out.println("int: x=" + x + " y=" + y);

        Integer p = 1, q = 2;
        swapInteger(p, q);
        System.out.println("Integer: p=" + p + " q=" + q);

        Kutu m = new Kutu(1), n = new Kutu(2);
        swapKutu(m, n);
```

```
        System.out.println("Kutu: m=" + m.deger + " n=" + n.deger);
    }
}
```

Çıktı:

```
kitap.ad = Degisti
int: x=1 y=2
Integer: p=1 q=2
Kutu: m=2 n=1
```

6.4 Nesneler Arası İşbirliği ve Etkileşim Diyagramları

Sonra nesneler arası ilişkiler ve işbirlikleri görülecek. Bunlar UML etkileşim diyagramıyla (interaction diagram), diğer adıyla sıralama diyagramıyla (sequence diagram) gösterilir. Bu konular teorik görünür ve çizim gerektirir. Hoca çizimlerin elle yapılmayacağını, bunun için araçlar olduğunu ve örnekleri birlikte çizeceklerini söyledi.

6.5 Kalıtım, Soyut Sınıf ve Arayüz

Ardından kalıtım, soyut sınıf (abstract class), arayüz (interface) ve çoklu kalıtım kavramlarına girilecek. Hoca önce "Java'da çoklu kalıtım yok" dediğini hatırlattı ve ekledi: Yok, ama arayüz mekanizmasıyla dolaylı yoldan sağlanabilir. Normalde kalıtım somut bir sınıf üzerinden gerçekleşir. Hoca, arayüzler ya da soyut sınıflar üzerinden de bunun yapılabildiğini söyledi.

> **Ders sonrası not:** Hoca çoklu kalıtımın arayüzler ya da soyut sınıflar üzerinden sağlanabildiğini söyledi; doğrusu yalnız arayüzler bunu sağlar. javac ile denendi: İki soyut sınıftan aynı anda türetme (`extends A, B`) hata verir. Bir soyut sınıftan türeyip iki arayüzü uygulayan `class Kitap extends Yayin implements Yazdirilabilir, Kaydedilebilir` satırı derlenir ve çalışır.

6.6 Çok Biçimlilik: İstanbulkart Örneği

Kalıtımla birlikte ortaya çıkan kavram çok biçimliliktir. Metotların ezilmesi (override) ve aşırı yüklenmesi (overload) bu başlıkta görülür. Hoca İstanbulkart örneğini verdi: Orada öğrenci, öğretmen, emekli, hepsi birer kullanıcıdır. Ama kart kullanıldığında birinin bakiyesinden 1 lira düşerken diğerinden 50 kuruş, bir başkasından 1,5 lira düşer. Aynı işlemin kullanıcı türüne göre farklı davranması çok biçimliliği ve metotların override edilmesini ortaya çıkarır. Hoca overload'ın benzer ama biraz farklı bir kavram olduğunu söyleyip konuyu sonraya bıraktı.

> **Ek bilgi (derste anlatılmadı):** Override'da alt sınıf, üst sınıfın aynı imzalı metodunu yeniden yazar ve hangi metodun çalışacağı nesnenin türüne göre belirlenir. Overload'da aynı adlı metotlar farklı parametre listeleriyle tanımlanır. javac ile denendi: `Kullanici k = new Ogrenci();` için `k.ucret()` çağrısı alt sınıfın metodunu çalıştırdı. `yaz(1)` ve `yaz("a")` çağrıları ise `int` ve `String` parametreli ayrı metotlara gitti.

6.7 Enumeration ve Jenerik Sınıflar

Geçen dönem C'de sınıf olarak olmayan biçimde kullanılan numaralandırmaların (enumeration) Java'daki sınıf biçimine bakılacak.

Sonra nesne yönelimli programlamayı üst seviyeye taşıyan jenerik sınıflar (generics) görülecek. Hocanın örneği sıralamadır. Bir sayı dizisi `int` ya da `long` olabilir ve onu sıralayan bir sınıf yazabilirsiniz. Ama aynı dizi `double` da olabilir. Hatta sayı olması da gerekmez: karakter, metin ya da bir nesne dizisi de sıralanabilir. Örneğin kitapları belli bir kritere göre sıralamak isteyebilirsiniz. Jenerik sınıflar bu durum için tasarlanır. Hocaya göre adı zor görünse de birkaç özelliği bilindikten sonra kolay bir konudur.

6.8 İstisna Yönetimi

C'de pek görmediğimiz ama programlamada çok kolaylık sağlayan bir mekanizma da istisna yönetimidir (exception handling).

Soru: En basit bildiğimiz istisna (exception) nedir?

Cevap: Öğrencilerden önce noktalı virgül eksikliği gibi bir cevap geldi. Hoca bunun sözdizimi hatası (syntax error) olduğunu, istisnadan farklı olduğunu söyledi. Doğru cevap sıfıra bölmedir (divide by zero). İstisna, programın çalışması sırasında (runtime) ortaya çıkan durumdur.

Programda bir istisna oluştuğunda programın kapanmasını istemeyiz. Programı kurduğunuz kişinin bilgisayarında bir şey olup programın kapanması ve kişinin onu yeniden açması kötü bir şeydir.

Kendi istisnalarımızı da tanımlayabiliriz. Hoca bunu, inanış gereği bazı ülkelerde 13 numaralı uçak ya da koltuk bulunmadığını duymasıyla örnekledi. Yani "şu veri gelirse bu bir istisnadır" diyebilir ve bunu işleyecek bir mekanizma kurabiliriz.

Hoca mekanizmayı Assembly'deki **kesmeye (interrupt)** benzetti. Olayın ne zaman olacağını bilmeyiz. Örneğin sıcaklık 25'e düşecek, ama ne zaman düşeceği belli değildir. Olayın olma anı, sıcaklığın 25'e düşmesidir. Burada da istisnanın ne zaman olacağı belli değildir. O olay için bir işleyici tasarlanır, istisna ile işleyici arasında bağlantı kurulur ve istisna olduğunda gereği yapılır. Sıfıra bölmede bu, "payda sıfır olamaz, geçerli bir veri girin" mesajı vermek olabilir.

Bunu C'de de yapabiliyorduk: `a` ve `b` alınır, `b` sıfırsa `do-while` döngüsüyle tekrar istenir. Ama hocaya göre C'de yapılan şey **istisnadan kaçınmadır (exception avoidance)**, yani hatanın oluşmasını engellemektir. Java'da her olasılığı önceden düşünmek zorunda değiliz. Genel istisna türleri de vardır ve onlardan yola çıkarak bir mekanizma tasarlanır.

İkinci örnek dizi sınırıdır. C'de 10 elemanlı bir dizide `a[10]`'a erişebilirsiniz. O adres başka bir sürece ait değilse işletim sistemi buna izin verir. Java ise işi güvence altına alır ve bir istisna üretir. Program da bu istisna üzerine mesaj verip hatayı gidermeye çalışır. Hoca istisnanın adını "array out of index boundary" diye andı.

> **Ders sonrası not:** Hoca Java'nın `a[10]` erişiminde derlenirken de doğrudan hata verdiğini söyledi; doğrusu sınır denetimi derleme zamanında yapılmaz. javac ile denendi: Aşağıdaki kod hatasız derlendi, hata çalışma sırasında `ArrayIndexOutOfBoundsException` olarak geldi. İstisnanın doğru adı da budur. `1 / b` işlemindeki sıfıra bölme ise `ArithmeticException` üretti.

```java
public class DiziSiniri {
    public static void main(String[] args) {
        int[] a = new int[10];
        a[9] = 5;
        System.out.println("a[9] = " + a[9]);
        try {
            a[10] = 7;
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Yakalandi: " + e.getMessage());
        }
        try {
            int b = 0;
            System.out.println(1 / b);
        } catch (ArithmeticException e) {
            System.out.println("Yakalandi: " + e.getMessage());
        }
        System.out.println("Program devam ediyor");
    }
}
```

Çıktı:

```
a[9] = 5
Yakalandi: Index 10 out of bounds for length 10
```

```
Yakalandi: / by zero
Program devam ediyor
```

6.9 Son Konular

Dönemin geri kalanında şunlar görülecek:

- Tip dönüşümleri (type casting)
- Dosyalar ve akışlar (stream)
- Çoklu iş parçacığı (multithreading): Java thread'leri üzerinde durulacak.
- İç sınıflar (inner class)
- Grafik kullanıcı arayüzü ve olay güdümlü programlama (event-driven programming)

Hoca, görsel programlamanın istense bir yıl boyunca ayrı ders olarak okutulabileceğini söyledi. Bu derste düğme, metin kutusu, radyo düğmesi ve açılır kutu (combo box) gibi kontrollerle bir arayüzün nasıl tasarlandığına bakılacak. Ayrıca arayüz ile arka plandaki nesne yönelimli tasarım arasında nasıl bağlantı kurulduğu anlatılacak. Bu kısım normalde programda yoktur. Ama hocaya göre bu bilgiler olmadan ders yarım kalmış gibi olur.

7. Neden Java?

Molanın ardından hoca Java'nın genel yapısına geçti. Nesne tabanlı programlama ve internet geliştikçe, birçok platformda çalışacak taşınabilir kod üretme ihtiyacı doğdu. Her platform için farklı bir yapı öğrenmeye gerek kalmasın isteniyordu. Java bu ihtiyaçtan ortaya çıkan dillerden biridir. C++ da bu ihtiyacı gören bir dildir.

Özellikle web sayfalarında grafik arayüz oluşturma gereksinimi çıkınca ve web sayfalarını programlamak masaüstüne göre zor olunca, Java hocaya göre 15-20 yıl önce çok güzel bir olanak sağladı. PHP ve ASP de vardı, ama onların da kendine göre zorlukları vardı. Java hem masaüstünde, hem web'de, hem de küçük cihazlarda çalışır. Ortaya çıkışıyla kabul gören dil hayatını bugün de sürdürmektedir.

8. Java Sürümleri: SE, ME ve EE

Java'nın farklı edisyonları vardır.

8.1 Standard Edition (SE)

Slaytta mobil cihazlar dışında birçok alan için olduğu yazılıydı. Bu edisyon daha çok masaüstü bilgisayarlarda çalışan ya da deneysel amaçlı programlar içindir. C dili gibi yalnız komut satırı üzerinden programlama imkânı sunar, ama grafik arayüz de tasarlanabilir. Uygulamalarımızda daha çok bunu kullanacağız.

8.2 Micro Edition (ME)

Akıllı telefon gibi küçük cihazlar üzerinde çalışmak için kullanılan edisyondur. Hoca bu noktada Android'in ilk dönemlerini anlattı. O zamanlar kodlama çoğunlukla Java ile yapılıyordu. Android Studio ve görsel araçlar henüz yaygın değildi. Grafik arayüz öğeleri XML biçiminde tasarlanıyor, sonra Java koduyla yönetiliyordu. Bu yüzden insanlar öğrenmekte zorlanıyordu.

> **Ders sonrası not:** Hoca bu tür Android uygulamalarının Micro Edition ile yapılabildiğini söyledi; doğrusu büyük olasılıkla Android, Java ME değil kendi SDK'sını kullanır.

8.3 Enterprise Edition (EE)

Kurumsal uygulamalar sunucu-istemci (client-server) mimarisine dayanır. Daha çok web tarafındaki uygulamalar burada geliştirilir. Arka uç (backend) denilen veri tabanına bağlanma ve algoritmik işlemler ile görsel arayüzlerin oluşturulması bu tarafta yapılır. Hoca burada Java

Server Pages'i (JSP) andı ve framework kavramının bir kütüphaneden çok daha kapsamlı olduğunu söyledi. Bir kullanıcı cep telefonundan ya da herhangi bir uygulamadan sunucuya bağlanır. Sunucudaki uygulamada Enterprise Edition ile yazılmış kodlar çalışır.

Hoca güncel örnek olarak Spring Framework'ü ve sunucu tarafında Apache Tomcat ile IBM WebSphere'i saydı. Bu başlığın açılması web programlama ve web tasarım konularına götürür.

> **Ders sonrası not:** Hoca Java Server Pages'ten bir framework olarak söz etti; doğrusu büyük olasılıkla JSP bir sayfa teknolojisidir, framework değildir.

8.4 Çok Katmanlı Uygulamalar

Slaytta geçen çok katmanlı uygulama (multi-tier application) şu demektir: Web ve mobil uygulamalarda mutlaka bir veri tabanı kullanılır. Projenin bir de iş mantığı (business logic) vardır. Ayrıca bir kullanıcı arayüzü bulunur. Her katman aslında Java'da yazılmış ayrı bir kod bölümüdür:

1. **Veri tabanı katmanı:** Veri tabanına bağlanır, sorgu gönderir, veri alır ve gönderir, ekleme, silme, güncelleme işlemlerini yapar.
2. **İş mantığı katmanı:** Veri tabanından bağımsız olarak işin mantığıyla ilgili işlemleri yapar.
3. **Kullanıcı arayüzü katmanı (user interface layer):** Tasarlanan grafik arayüzle ilgilenir.

Hoca e-ticaret uygulamalarını bu yapıya örnek gösterdi.

Bunlara başka web servisleri de eklenebilir. Bir web sayfası başka bir yerden servis alabilir. Hocanın örneği E-Devlet'ti:

1. E-Devlet'e girip öğrenci belgesi almak istersiniz.
2. Bütün öğrencilerin bilgisi E-Devlet'in sunucusunda değildir. E-Devlet burada kullanıcı için bir arayüz katmanıdır.
3. E-Devlet, Yıldız Teknik Üniversitesi'nin öğrenci işleri sunucusuna bağlanır.
4. İki sunucu arasında bir servis işlemi gerçekleşir.

Hocanın özetiyle, web programlama gibi işler Enterprise Edition tarafında yapılır.

9. Tek Sözdizimi, Güçlü Dokümantasyon

Bu edisyonların hepsinde yazdıklarımız C sözdizimine çok benzer. `for` döngüsü her yerde aynı yazılır. Ama birinde küçük bir cihazı ya da telefonu işletirken, diğerinde sunucu üzerinde işlem yaparsınız. Her alan için farklı bir dil öğrenmeye gerek kalmaz. Hocaya göre bu dilin en güzel yanlarından biridir. Java'nın başka bir özelliği de sürekli kendini güncellemesi ve yeni şeyler çıkarmasıdır. Dil Oracle tarafından satın alınmıştır.

Hocaya göre C'ye kıyasla bir üstünlüğü de dokümantasyondur. C güçlü bir dildir, ama dokümantasyon ve uyum konusunda bazı sorunları vardır. Java'da ise bu kurumsal olarak oturmuştur. Hoca bunu bir benzetmeyle anlattı: Bir transistörün, direncin ya da entegre devrenin veri sayfası (data sheet) nasıl varsa, kullanacağınız sınıfların da kendi "veri sayfaları" vardır. Hazır yapıları kullanırken neyin neyle nasıl ilişkide olduğunu bilmeniz gerekir. Bunu bilmeden kod yazmak mümkün değildir. Hoca aynı düzenin C#'ta da olduğunu, açık kaynak dünyasında Python gibi dillerde ise henüz oturmadığını düşündüğünü söyledi.

10. Java Programı Nasıl Derlenir ve Çalışır?

10.1 Kaynak Koddan Bytecode'a

Kaynak kodumuz `.java` uzantılı bir dosyadır ve C'ye çok benzer. Bir programın çalıştırılabilir hale gelmesi birkaç aşamadan oluşur:

1. **Derleme:** Java derleyicisi (Java compiler) `.java` dosyasını alır ve bir ara kod dosyasına dönüştürür. Bu dosyanın uzantısı `.class`'tır. İçindeki koda object code ya da **bytecode** denir.

2. **Taşıma:** Aynı `.class` dosyası, işletim sistemi farklı da olsa her yerde çalıştırılabilir durumdadır.
3. **Çalıştırma:** Java Virtual Machine (JVM), bulunduğu işletim sistemine ve işlemciye bağlı olarak bytecode'u işlemcinin anlayacağı makine koduna dönüştürür.

Hoca, `.java` dosyasını not defteriyle açınca kendi yazdığımız kodu göreceğimizi, `.class` dosyasını açınca ise baytlardan oluşan anlaşılmayan bir kod göreceğimizi söyledi. C'de derleme aşamasında Assembly kodları görebiliyorduk. Burada onu doğrudan göremeyiz, yapı biraz farklıdır.

> **Ek bilgi (derste anlatılmadı):** Python ile okunan `Merhaba.class` dosyasının ilk 4 baytı `cafebabe` çıktı. `.class` dosyasının bayt düzeyinde kendi biçimi olduğunu gösteren küçük bir örnektir.

JDK kurulumu işletim sistemine göre değişir. Windows, Mac ve Linux için farklı indirmeler vardır. Ama hepsinde derleyici aynı şekilde çalışır ve aynı bytecode'u üretir. Önemli olan sonraki kısımdır: `.class` dosyası her yerde çalışır.

Bunun bir şartı vardır. Kullanılan kütüphanenin o platformda desteklenmesi gerekir. İki sayıyı toplayan bir program Mac'te de Windows'ta da çalışır. Ama kullandığınız bir Java kütüphanesi örneğin Mac'te desteklenmiyorsa program orada çalışmaz.

10.2 JDK, JRE ve JVM

Hoca üç kavramı şöyle ayırdı:

- **Java Development Kit (JDK):** Geliştiriciler içindir. Program yazacaksanız bunu kurmanız gerekir, çünkü kodu derleyip `.class` dosyasını oluşturmanız gerekir.
- **Java Runtime Environment (JRE):** Son kullanıcılar içindir. `.class` dosyasını alıp çalıştıracak mekanizmayı sağlar.
- **Java Virtual Machine (JVM):** Bytecode'u bulunduğu makineye göre işleten mekanizmadır.

JDK kurulduğunda JRE de içine dahil edilir, çünkü geliştiricinin programını test etmesi gerekir.

Hoca çalıştırma tarafında iki dosyadan bahsetti: `java.exe` komut satırı programları için, `javaw.exe` ise GUI programları içindir.

> **Ders sonrası not:** Hoca derleyicinin "tam bir derleyici olmadığını, yorumlayıcıyı da içinde barındırdığını" ve JVM'nin `.jar` uzantılı "executable dosya oluşturduğunu" söyledi; doğrusu derleyici yalnız derler, `java` komutu da çalıştırırken dosya üretmez. javac ile denendi: `javac Merhaba.java` programı çalıştırmadan yalnız `Merhaba.class` üretti. `java Merhaba` programı çalıştırdı ve klasörde yeni dosya oluşmadı. `.jar` dosyası ise ayrı bir `jar` aracıyla oluştu ve içinde `.class` dosyası ile `META-INF/MANIFEST.MF` listelendi. `java -XX:+PrintCompilation` çıktısında sık çağrılan metodun çalışma sırasında derlendiği görüldü, yani JVM'nin makine koduna çevirmesi bellekte olur.

```java
public class Merhaba {
    public static void main(String[] args) {
        System.out.println("Merhaba Java");
    }
}
```

```
javac Merhaba.java                Merhaba.class oluşur, program çalışmaz
java Merhaba                      "Merhaba Java" yazar, yeni dosya oluşmaz
jar cf merhaba.jar Merhaba.class  merhaba.jar arşivi oluşur
```

> **Ders sonrası not:** Hoca `javaw.exe`'nin `.class` dosyasını "executable hale dönüştürdüğünü" söyledi; doğrusu büyük olasılıkla `java.exe` ile `javaw.exe` arasındaki fark, `javaw`'ın konsol penceresi açmadan çalıştırmasıdır. Windows'ta denenmedi.

10.3 Komut Satırı ve IDE

Komut satırından Java kodu derlenebilir. Önce Java derleyicisi çalıştırılır ve dosyanın yolu verilir. Derleyici `.class` dosyasını oluşturur. Sonra `java` ile program çalıştırılır. Bu, birbirini takip eden bir süreçtir. Ama bunların hepsini bir arada yapan bir geliştirme ortamına (IDE) ihtiyaç vardır.

11. Java'nın Gelişimi

Hoca slaytta Java'nın Java 1.0 ve 1.1 ile başlayıp Java Standard Edition 15'e kadar gelen sürüm tablosunu gösterdi. Daha yeni sürümler olabileceğini de ekledi.

> **Ders sonrası not:** Hoca güncel sürümün 15 olduğunu, bakılması gerektiğini söyledi. Büyük olasılıkla ders tarihinde (10 Mart 2021, tarih kısa nottan) güncel sürüm Java SE 15 idi ve Java SE 16 birkaç gün sonra çıktı.

11.1 Arayüz Örneği: Bilgi Nasıl Eskir?

Hoca sürümlerin neden önemli olduğunu bir anıyla anlattı. Birkaç yıl önce bu dersin asistanıyken arayüzlerdeki metotlara gövde yazılamayacağını öğretiyorlardı.

Soru: Fonksiyon prototipi ne demekti?

Cevap: Bir öğrenci, yalnız fonksiyon adı, dönüş tipi ve parametre tiplerini içeren yapı olduğunu söyledi. Hoca onayladı: Dönüş tipi, parametreler ve fonksiyon adı prototipte olmazsa olmazdır. Gerçekleme (implementation) ise prototipte yoktur.

Arayüzler uzun süre yalnız prototip içeriyordu. Hoca sonra güncellemeleri incelerken Oracle'ın sayfasında `default` anahtar kelimesiyle arayüz içinde bir metodun gövdesinin yazılabildiğini gördü. Önceden "yazılmaz, hata verir" denen şey, belli bir sürümden sonra geçersiz olmuştur. Hoca bunun sürüm 8 ile geldiğini zannettiğini söyledi.

> **Ders sonrası not:** javac ile denendi: Aşağıdaki kod `-source 8` seçeneğiyle derlendi ve çalıştı, yani `default` metotlar Java 8 dil düzeyinde vardır. javac 25 sürüm 7 düzeyini desteklemediği için önceki sürüm denenemedi. Büyük olasılıkla bu özellik Java 8 ile gelmiştir.

```
interface Yazdirilabilir {
    void yazdir();                      // yalniz prototip
    default void baslik() {             // default ile govde
        System.out.println("--- baslik ---");
    }
}

interface Kaydedilebilir {
    void kaydet();
}

abstract class Yayin { }

class Kitap extends Yayin implements Yazdirilabilir, Kaydedilebilir {
    public void yazdir() { System.out.println("Kitap yazdirildi"); }
    public void kaydet() { System.out.println("Kitap kaydedildi"); }
}

public class ArayuzDeneme {
    public static void main(String[] args) {
        Kitap k = new Kitap();
        k.baslik();
        k.yazdir();
        k.kaydet();
    }
}
```

Çıktı:

```
--- baslik ---
Kitap yazdirildi
Kitap kaydedildi
```

Bunun sebebi, gereksinimlerin değişmesi ve programlama dillerinin birbirlerinden etkilenip özelliklerini benimsemesidir. Bu yüzden sürekli bir gelişim vardır.

Slayttaki tabloda Java 8'in eski sistemler için uzun vadeli destek aldığı da görülür. Yeni sürümler yaklaşık 6 ayda bir çıkar, çünkü dil büyük veri, çoklu iş parçacığı ve web programlama gibi yeni gelişmelere açıktır. Dile eklenen bir şey bütün dili etkiler. Bu değişiklikler de belli aralıklarla yeni sürümler olarak gelir. Dersin slaytları Java 1.5'e göre hazırlanmıştır. Hoca eskimiş (deprecated) kısımların yenisini söyleyeceğini belirtti.

11.2 Grafik Arayüz Kütüphaneleri

Hoca üç kütüphaneyi ekranda göstererek karşılaştırdı:

1. **AWT:** Java'nın ilk kullandığı grafik arayüz kütüphanelerinden biridir. Görünümü Windows 98'i andırır, bazı eski programlarda karşımıza çıkar.
2. **Swing:** Görsel olarak daha güzel öğeler içeren, daha zengin bir kütüphanedir.
3. **JavaFX:** Günümüzde kullanılan, hem tasarım hem programlama açısından daha zengin bir kütüphanedir. Fark yalnız renkli düğmeler değildir, işlevsellik açısından da kolaylıklar sunar. Derste kısmen JavaFX'e de girilecek.

Hoca ayrıca Java'nın büyük veri tarafında da uygulamaları olduğunu, Hadoop'un Java ile yazıldığını bildiğini söyledi. Micro Edition ve web programlama tarafında da geniş bir uygulama yelpazesi vardır. Yine de Java'nın her nesne tabanlı özelliği henüz sağlamadığını ekledi.

12. Geliştirme Ortamları, UML Araçları ve Plugin Kavramı

12.1 IDE Seçimi

Java'nın sitesinden istediğiniz sürümü işletim sisteminize göre indirebilirsiniz. Bununla birlikte JRE ve JDK da gerekir. Hoca sitenin bunları toptan indirdiğini söyledi.

Hoca yıllardır **NetBeans** kullandığını, bazı işlemleri Eclipse'e göre daha kolay bulduğunu söyledi. Amaç aynı şeyleri tekrar tekrar zor yoldan yapmak değil, hedefe hızlı gitmektir. Diğer hocalar **Eclipse** de kullanıyor. Bir öğrencinin önerdiği **IntelliJ IDEA** da kullanılabilir. Hoca bunu çok duyduğunu ama kullanmadığını söyledi.

> **Ders sonrası not:** Hoca NetBeans'in Oracle'ın sayfasından indirilebildiğini söyledi; doğrusu büyük olasılıkla NetBeans'in güncel sürümleri Apache NetBeans adıyla Apache Yazılım Vakfı üzerinden dağıtılıyor.

Eclipse taşınabilirdir (portable), kurulum yapmadan doğrudan açıp kullanabilirsiniz. NetBeans ise "İleri, İleri" diyerek kurulur ve kendi içinde bir GUI editörü vardır. İki ortamda da yalnız Java değil, C, C++ ve hocanın zannına göre PHP kodu da çalıştırılabilir. Hoca bunların hata ayıklama (debug) imkânlarının Dev-C++ ve Code::Blocks'a göre çok daha iyi olduğunu söyledi.

Soru: NetBeans mi, Eclipse mi kullanılacak?

Cevap: Tercih size bağlıdır. Hoca NetBeans kullanacağını söyledi. Ne kadar çok şey öğrenirseniz o kadar kârdır.

12.2 UML Araçları

Dersin ikinci boyutu olan nesneye dayalı kavramlar için UML sınıf diyagramlarıyla çok uğraşacağız. Hoca şu ücretsiz araçları andı:

- **ArgoUML:** İstediğiniz dili (C++, Java, C#) seçip tasarımdan kod üretebiliyorsunuz (code generator).
- **Violet UML:** Yalnız çizim yapılabilen basit bir araçtır.
- **NetBeans ve Eclipse eklentileri:** Bu araçların işlevini ortamın içine getirirler.

Hoca kendi NetBeans'inde UML eklentisini aradı ama bulamadı. Bilgisayarını yeniden kurduğunda eklemeyi unutmuş olabileceğini söyledi.

12.3 Plugin Nedir?

Soru: Plugin nedir, ne biliyorsunuz?

Cevap: Bir öğrenci, genellikle opsiyonel olan, kullanıcıların da üretebildiği yardımcı program parçaları olduğunu söyledi. Hoca onayladı ve Zoom derslerini indirmek için Chrome ya da Firefox'a kurulan eklentiyi örnek verdi.

Hocanın tanımıyla plugin şudur: Geliştirilmiş bir uygulamada **açık bir kapı** bırakırsınız ve "bu kapı sayesinde yeni eklentiler yapılabilir" dersiniz. Bu aslında bir tasarım kalıbıdır. Hoca imkân olursa bunu Java'da komut satırından çalışan basit bir programla göstereceğini söyledi: Önce program geliştirilecek, sonra ona plugin yazılacak.

Plugin özellikle açık kaynak uygulamalarda önemlidir. Sizin düşünemediğinizi başkası düşünüp ekler ve uygulamayı zenginleştirir. Bu hem size hem de uygulamayı geliştiren kişiye faydalıdır. NetBeans, Eclipse ve Visual Studio'da birçok eklenti imkânı vardır. Örneğin Eclipse'e UML çizmek için ya da GUI uygulamaları için eklenti kurulabilir.

> **Hocanın vurgusu:** Bu işin asıl zor olan tarafı nesne tabanlı analiz ve tasarımdır. Hangi sınıflar olacak, hangi nesneler olacak, bu nesneler arasındaki ilişkiler nasıl olacak?

13. IntelliJ IDEA Üzerinde Kısa Bir İnceleme

Ders sonunda bir öğrenci ekranını paylaşarak IntelliJ IDEA'yı tanıttı. Hoca ortamı dersin ihtiyaçları açısından sorularla inceledi. Baktığı noktalar, bir IDE seçerken neye bakılacağını da gösterir:

1. **Proje açma:** Var olan bir proje açıldığında kaynak kod ve sınıflar birlikte görünüyor.
2. **Hata ayıklama:** Hoca kesme noktası (breakpoint) koymayı sordu. Satır numaralarının yanında bir yer arandı, ekranda "Set Bookmark" gibi bir seçenek görüldü, ama konu netleşmedi.
3. **GUI desteği:** Yeni proje oluştururken JavaFX seçeneği görüldü. Swing için bir seçenek arandı. Bir öğrenci NetBeans'te Swing desteğinin kaldırıldığını duyduğunu söyledi. Hoca bunu kendisinin de kontrol edeceğini belirtti.
4. **Plugin listesi:** File menüsündeki Settings altında Plugins bölümü bulundu. "UML" diye aratınca **PlantUML Integration** ve **Simple UML** çıktı. "Swing" diye aratınca hocanın ihtiyacını tam karşılayan bir sonuç çıkmadı.

Listede geçen AWS sözcüğünün Amazon Web Services olduğunu, Java'daki AWT ile karışabileceğini de hoca belirtti. AWT çok eski bir kütüphanedir ve çok kullanılmayacaktır. Hoca ayrıca IntelliJ'in PyCharm'a çok benzediğini fark etti.

Sonuç olarak hoca IntelliJ'i fena bulmadı, ama uygulama saatine kadar araştıracağını söyledi. İsteyen istediği ortamı kullanabilir. Hocanın NetBeans tercihi biraz da alışkanlıktır.

Dersin Toparlaması

Bu derste Java kodu yazılmadı. Hocanın da birkaç kez dikkat çektiği gibi ağırlık, dilin kendisinden önce düşünme biçimindeydi. Yapısal programlamada fonksiyonlar ve dosyalar büyük bir projeyi yönetmeye yetmez. Nesne yönelimli yaklaşım; zaman, yönetim, bakım ve anlaşılabilirlik kazancı sağlar. Yazılımın yaşayabilir kalmasına, yani yeni modüllerin sistemi bozmadan takılabilmesine imkân verir. Asıl zor olan da Java'yı kodlamak değil, nesneleri ve aralarındaki ilişkileri doğru analiz edip tasarlamaktır. Bu tasarımın ortak dili UML'dir.

Java tarafında ise şu temel görüldü: `.java` kaynak kodu derleyiciyle `.class` bytecode'una çevrilir ve JVM bunu bulunduğu makinede çalıştırır. JDK geliştirici, JRE kullanıcı içindir. SE, ME ve EE aynı sözdizimiyle farklı alanlara hitap eder. Hoca sonraki derste sınıfın ve nesnenin ne olduğuna, birbirlerinden nasıl ayrıldıklarına bakılacağını söyledi. Uygulama saatinde de NetBeans kullanımı, Hello World yazma, derleme ve hata ayıklama gösterilecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.